Microsoft AutoGen: The Complete Course

Build Multi-Agent AI Systems with Microsoft's Open-Source Framework

---

Course Overview

Microsoft AutoGen is an open-source framework that enables developers to build applications powered by multiple AI agents that can communicate and collaborate to solve complex tasks. Instead of working with a single LLM, AutoGen lets you orchestrate multiple specialized agents—each with distinct roles, tools, and behaviors—that work together through structured conversations.

This course will take you from understanding the core concepts to building production-ready multi-agent systems.

Prerequisites: - Python 3.8+ installed - Basic understanding of LLMs (APIs, prompts, tokens) - Familiarity with async programming concepts (helpful but not required) - An OpenAI API key (or compatible endpoint like Azure OpenAI, Anthropic, etc.)

What you'll build: - A code assistant multi-agent system - A research team of collaborating agents - A human-in-the-loop workflow - Production-ready agent patterns

---

Module 1: Introduction to AutoGen

What is Microsoft AutoGen?

AutoGen is a framework for building **multi-agent applications** where multiple AI agents communicate with each other to accomplish tasks. Think of it as giving you the infrastructure to create a team of AI workers, each with specific responsibilities, that can collaborate without constant human intervention.

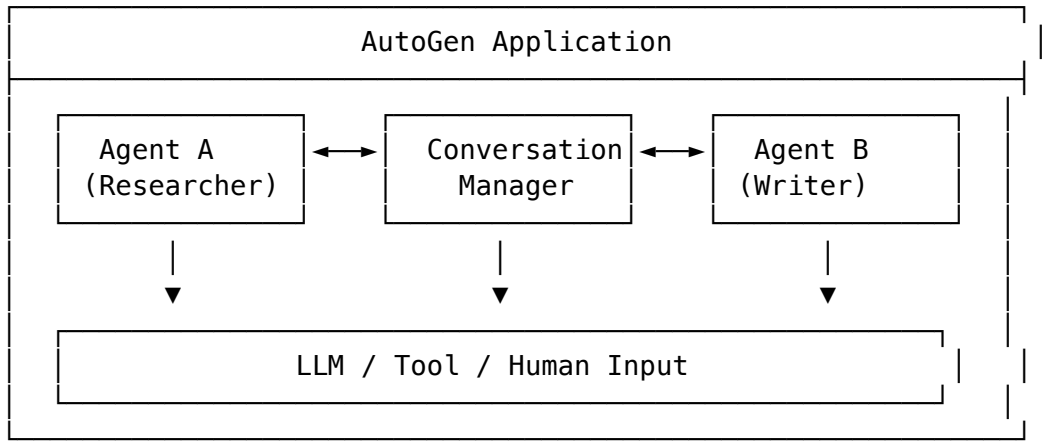
Key capabilities: - **Multi-agent conversations**: Agents talk to each other in a structured way - **Flexible agent roles**: Create assistants, users, executors, or custom roles - **Tool/function calling**: Agents can execute code, call APIs, or run custom functions - **Human-in-the-loop**: Pause workflows for human input when needed - **Persistent state**: Conversations maintain context across turns

Why Multi-Agent Systems?

Single LLM calls have limitations: - Limited context windows - No persistent memory across sessions - Difficulty handling complex, multi-step tasks - Hard to specialize for different aspects of a problem

Multi-agent architectures solve these by: - **Dividing labor**: Each agent specializes in one domain - **Parallel processing**: Agents can work simultaneously - **Robustness**: One agent's failure doesn't halt everything - **Scalability**: Add more agents for more complex workflows

AutoGen Architecture Overview



Module 2: Setting Up Your Environment

Installation

```
# Create a virtual environment (recommended)
python -m venv autogen-env
source autogen-env/bin/activate  # On Windows: autogen-
        env\Scripts\activate

# Install AutoGen
pip install pyautogen

# Install additional dependencies you'll likely need
pip install openai dotenv
```

Configuration

AutoGen requires configuration for the LLM backends. Create a `O_CONFIG.json` file:

```
{
  "model": "gpt-4",
  "api_key": "your-openai-api-key"
}
```

Or use environment variables:

```
export OPENAI_API_KEY="your-api-key-here"
```

For Azure OpenAI:

```
{
  "model": "gpt-4",
  "api_type": "azure",
  "api_base": "https://your-resource.openai.azure.com/",
  "api_key": "your-azure-key",
  "api_version": "2024-02-01"
}
```

Quick Verification

Create a test script to verify your setup:

```
import autogen

# Simple configuration
config_list = [
    {
        "model": "gpt-4",
        "api_key": "your-api-key"
    }
]

# Create a simple agent
assistant = autogen.AssistantAgent(
    name="test_agent",
    llm_config={"config_list": config_list}
)

print("✓ AutoGen installed successfully!")
```

Module 3: Core Concepts

3.1 The Agent Base Class

Every agent in AutoGen inherits from a base class. The two most common:

AssistantAgent - An AI-powered agent that responds to messages using an LLM

```
assistant = autogen.AssistantAgent(
    name="assistant",
    llm_config={"config_list": config_list},
    system_message="You are a helpful AI assistant."
)
```

UserProxyAgent - Represents a human user; can execute code and functions

```
user_proxy = autogen.UserProxyAgent(
    name="user_proxy",
    human_input_mode="NEVER"  # Options: ALWAYS, NEVER, TERMINATE
)
```

3.2 Conversations

Agents communicate through **conversations**—structured exchanges of messages:

```
# Initiate a conversation
user_proxy.initiate_chat(
    assistant,
    message="Write a Python function to calculate fibonacci numbers."
)
```

3.3 Messages

Messages in AutoGen have a specific structure:

```
# Message structure
message = {
    "role": "assistant",      # or "user", "system"
    "content": "The response text",
    "name": "agent_name",    # optional
    "tool_calls": []         # for function calls
}
```

3.4 LLM Configuration

The `llm_config` dictionary controls how the LLM behaves:

```
llm_config = {
    "config_list": config_list,      # List of model configurations
    "temperature": 0.7,              # Creativity vs determinism (0-1)
    "max_tokens": 2000,             # Maximum response length
    "cache_seed": None,             # For caching responses (None = no
        cache)
    "functions": [                  # Function definitions
        {
            "name": "calculate",
            "description": "Perform a calculation",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {"type": "string"}
                },
                "required": ["expression"]
            }
        }
    ]
}
```

Module 3 Summary

- **AssistantAgent** is your AI-powered worker
- **UserProxyAgent** represents users and can execute code
- Conversations are initiated with `initiate_chat()`
- LLM configuration controls model behavior and tools

Module 4: Building Your First AutoGen Application

A Simple Two-Agent Conversation

Let's build a basic code assistant:

```python
import autogen
from autogen import AssistantAgent, UserProxyAgent

# Configuration
config_list = [{"model": "gpt-4", "api_key": "your-api-key"}]

# Create agents
assistant = AssistantAgent(
    name="code_assistant",
    llm_config={"config_list": config_list},
    system_message="You are a Python expert. Write clean, well-
        documented code."
)

user_proxy = UserProxyAgent(
    name="human_user",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=10
)

# Start conversation
user_proxy.initiate_chat(
    assistant,
    message="Create a Python class for managing a simple inventory
        system with add, remove, and list operations."
)
```

Running the Code

Save as `first_app.py` and run:

```
python first_app.py
```

You'll see the agents exchange messages, with the assistant providing code and the user proxy (in NEVER mode) automatically responding.

Understanding the Flow

```
┌──────────────┐                    ┌──────────────────┐
│UserProxy     │── "Create a class..." │                  │
│  Agent       │                    │   Assistant      │
│              │◄── Python code     │    Agent         │
└──────────────┘                    │                  │
                                    └──────────────────┘
```

Exercise 1

Create a two-agent system where: 1. One agent is a "code reviewer" that provides feedback 2. The other is a "code writer" that generates code 3. Have them work together to produce a solution

---

Module 5: Working with Different Agent Types

5.1 AssistantAgent - The AI Worker

The primary agent for LLM-powered responses:

```
assistant = AssistantAgent(
    name="writer",
    llm_config={
        "config_list": config_list,
        "temperature": 0.8,  # More creative
    },
    system_message="""You are a technical writer.
    - Write clear, concise documentation
    - Use code examples when helpful
    - Explain complex concepts simply"""
)
```

5.2 UserProxyAgent - The Human Interface

Options for human input mode:

```
# Option 1: Never ask for input - auto-continue
user_proxy = UserProxyAgent(
    name="auto_user",
    human_input_mode="NEVER"
)

# Option 2: Always ask for input
user_proxy = UserProxyAgent(
    name="manual_user",
    human_input_mode="ALWAYS"
)

# Option 3: Ask only when needed (terminator keywords)
user_proxy = UserProxyAgent(
    name="hybrid_user",
    human_input_mode="TERMINATE"
)
```

5.3 GroupChat - Multiple Agents

Orchestrate multiple agents in a group conversation:

```
from autogen import GroupChat, GroupChatManager

# Create multiple agents
```

```
agent1 = AssistantAgent(name="researcher", llm_config={"config_list":
        config_list})
agent2 = AssistantAgent(name="writer", llm_config={"config_list":
        config_list})
agent3 = AssistantAgent(name="reviewer", llm_config={"config_list":
        config_list})

# Create group chat
groupchat = GroupChat(
    agents=[agent1, agent2, agent3],
    messages=[],
    max_round=10
)

# Create manager
manager = GroupChatManager(
    groupchat=groupchat,
    llm_config={"config_list": config_list}
)

# Initiate group conversation
user_proxy.initiate_chat(
    manager,
    message="Research the benefits of microservices and write a blog
        post about it."
)
```

5.4 Custom Agent Registration

Register custom reply functions:

```
def custom_reply(recipient, messages, sender, config):
    """Custom logic to determine responses"""
    last_message = messages[-1]["content"]

    if "error" in last_message.lower():
        return False, "I detected an error. Let me fix that."

    # Return (should_continue, response)
    return True, None  # Continue with normal LLM response

assistant.register_reply(custom_reply)
```

Module 5 Summary

- **AssistantAgent**: AI-powered responses via LLM
- **UserProxyAgent**: Human interaction and code execution
- **GroupChat**: Multiple agents in one conversation
- **register_reply()**: Custom response logic

Module 6: Tools and Function Calling

6.1 Defining Functions

Functions give agents capabilities beyond text generation:

```
# Define a function the agent can call
def get_weather(location: str, unit: str = "celsius") -> str:
    """Get weather for a location.

    Args:
        location: City name
        unit: celsius or fahrenheit
    """
    # In production, call a weather API
    return f"The weather in {location} is 22°C"

# Register with the agent
assistant = AssistantAgent(
    name="weather_assistant",
    llm_config={
        "config_list": config_list,
        "functions": [
            {
                "name": "get_weather",
                "description": "Get current weather for a location",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "location": {"type": "string", "description":
        "City name"},
                        "unit": {"type": "string", "enum": ["celsius",
        "fahrenheit"]}
                    },
                    "required": ["location"]
                }
            }
        ]
    }
)

# Register the function
assistant.register_function(
    function_map={"get_weather": get_weather}
)
```

6.2 Code Execution

UserProxyAgent can execute code:

```
# Enable code execution
user_proxy = UserProxyAgent(
    name="code_executor",
    human_input_mode="NEVER",
    code_execution_config={
        "work_dir": "coding",
        "use_docker": True  # Run in isolated container
```

```
    }
)
```

6.3 Complete Tool-Using Example

```python
import autogen
from autogen import AssistantAgent, UserProxyAgent

config_list = [{"model": "gpt-4", "api_key": "your-api-key"}]

# Define functions
def calculate(expression: str) -> float:
    """Evaluate a mathematical expression."""
    try:
        result = eval(expression, {"__builtins__": {}}, {})
        return f"Result: {result}"
    except Exception as e:
        return f"Error: {str(e)}"

def get_stock_price(symbol: str) -> dict:
    """Get current stock price (mock)."""
    prices = {"AAPL": 178.50, "GOOGL": 140.25, "MSFT": 378.90}
    return {"symbol": symbol, "price": prices.get(symbol, "Unknown")}

# Create assistant with functions
assistant = AssistantAgent(
    name="financial_assistant",
    llm_config={
        "config_list": config_list,
        "functions": [
            {
                "name": "calculate",
                "description": "Evaluate mathematical expressions",
                "parameters": {
                    "type": "object",
                    "properties": {"expression": {"type": "string"}},
                    "required": ["expression"]
                }
            },
            {
                "name": "get_stock_price",
                "description": "Get current stock price",
                "parameters": {
                    "type": "object",
                    "properties": {"symbol": {"type": "string"}},
                    "required": ["symbol"]
                }
            }
        ]
    }
)

# Register functions
assistant.register_function(
    function_map={
        "calculate": calculate,
        "get_stock_price": get_stock_price
```

```
    }
)

# Create user proxy
user_proxy = UserProxyAgent(
    name="user",
    human_input_mode="NEVER"
)

# Test it
user_proxy.initiate_chat(
    assistant,
    message="What's the current price of AAPL, and what is 150 * 25 +
        300?"
)
```

Module 6 Summary

- Define functions in `llm_config["functions"]`
- Register function implementations with `register_function()`
- UserProxyAgent can execute Python code automatically
- Functions enable agents to take real-world actions

Module 7: Advanced Conversation Patterns

7.1 Nested Chats

Agents can initiate sub-conversations with other agents:

```
# Agent that can delegate to specialists
manager = AssistantAgent(
    name="manager",
    llm_config={"config_list": config_list},
    system_message="You coordinate projects. Delegate to specialists."
)

researcher = AssistantAgent(
    name="researcher",
    llm_config={"config_list": config_list},
    system_message="You research topics thoroughly."
)

writer = AssistantAgent(
    name="writer",
    llm_config={"config_list": config_list},
    system_message="You write clear, engaging content."
)

# Register nested chat capability
def research_and_write(task):
    # Initiate research
    researcher.initiate_chat(
        manager,
        message=f"Research: {task}"
    )
```

```
    # Get research results and write
    writer.initiate_chat(
        manager,
        message=f"Write about: {researcher.chat_messages}"
    )

# Or use the built-in nested chat pattern:
assistant = AssistantAgent(
    name="coordinator",
    llm_config={"config_list": config_list}
)

assistant.register_nested_chat(
    chat_queue=[researcher, writer]
)
```

7.2 Sequential Chats

Chain conversations together:

```
# Chat 1: Research
user_proxy.initiate_chat(
    assistant,
    message="Research the history of artificial intelligence."
)

# Chat 2: Build on previous context
user_proxy.initiate_chat(
    assistant,
    message="Now summarize the key points for a general audience."
)
```

7.3 Speaker Selection in GroupChat

Control who speaks next:

```
groupchat = GroupChat(
    agents=[researcher, writer, reviewer],
    messages=[],
    max_round=15,
    speaker_selection_method="round_robin"  # Or "auto" or custom
        function
)

# Custom speaker selection
def select_speaker(last_speaker, selector):
    """Custom logic for who speaks next."""
    messages = selector.chat_messages
    if len(messages) < 3:
        return researcher
    return writer

groupchat = GroupChat(
    agents=[researcher, writer, reviewer],
    messages=[],
    speaker_selection_method=select_speaker
)
```

7.4 Conditional Responses

```python
def conditional_reply(recipient, messages, sender, config):
    """Reply based on message content."""
    last_msg = messages[-1]["content"]

    if "?" in last_msg:
        return True, None  # Let LLM handle questions

    if len(last_msg) > 1000:
        return True, "That's a lot of text. Let me summarize..."

    return True, None  # Default: continue normally

assistant.register_reply(conditional_reply)
```

Module 8: Human-in-the-Loop Patterns

8.1 Always Ask for Human Input

```python
user_proxy = UserProxyAgent(
    name="human_in_loop",
    human_input_mode="ALWAYS",
    max_consecutive_auto_reply=3
)

# When running, the agent will pause and wait for your input
user_proxy.initiate_chat(
    assistant,
    message="Write a function to sort a list."
)
```

8.2 Terminate with Human Approval

```python
user_proxy = UserProxyAgent(
    name="approver",
    human_input_mode="TERMINATE",
    is_termination_msg=lambda x: x.get("content",
        "").find("TERMINATE") >= 0
)

# The agent can send "TERMINATE" to stop the conversation
assistant = AssistantAgent(
    name="assistant",
    llm_config={"config_list": config_list},
    system_message="When the task is complete, respond with
        'TERMINATE'"
)
```

8.3 Selective Human Intervention

```python
import autogen

def should_interrupt(recipient, messages, sender, config):
```

```
    """Determine if human input is needed."""
    last_message = messages[-1]["content"]

    # Interrupt for sensitive operations
    sensitive_words = ["delete", "destroy", "drop table", "rm -rf"]
    return any(word in last_message.lower() for word in
        sensitive_words)

user_proxy = UserProxyAgent(
    name="guardian",
    human_input_mode="TERMINATE"
)

user_proxy.register_reply(
    trigger=should_interrupt,
    reply_func=lambda *args: (True, "Human approval needed for this
        action.")
)
```

8.4 Example: Human Approval Workflow

```
import autogen
from autogen import AssistantAgent, UserProxyAgent

config_list = [{"model": "gpt-4", "api_key": "your-api-key"}]

# Agent that suggests actions
planner = AssistantAgent(
    name="planner",
    llm_config={"config_list": config_list},
    system_message="You plan tasks. When you suggest an action, end
        with 'APPROVE?'"
)

# Human reviewer
reviewer = UserProxyAgent(
    name="reviewer",
    human_input_mode="TERMINATE",
    is_termination_msg=lambda x: x.get("content", "").find("APPROVE?")
        >= 0
)

# Custom reply to handle approval
def handle_approval(recipient, messages, sender, config):
    last_msg = messages[-1]["content"]
    if "APPROVE?" in last_msg:
        return True, "Waiting for human approval..."
    return True, None

planner.register_reply(handle_approval)

# Run
reviewer.initiate_chat(
    planner,
    message="Plan a deployment strategy for a web application."
)
```

Module 9: Real-World Use Cases

9.1 Code Review System

```python
import autogen
from autogen import AssistantAgent, UserProxyAgent, GroupChat,
        GroupChatManager

config_list = [{"model": "gpt-4", "api_key": "your-api-key"}]

# Define agents
code_reviewer = AssistantAgent(
    name="code_reviewer",
    llm_config={"config_list": config_list},
    system_message="""You review code for:
    - Bugs and security issues
    - Code quality and readability
    - Performance optimization opportunities
    - Best practices compliance"""
)

code_fixer = AssistantAgent(
    name="code_fixer",
    llm_config={"config_list": config_list},
    system_message="""You fix issues identified in code review.
    Apply fixes accurately and preserve the original functionality."""
)

# Group chat for review workflow
groupchat = GroupChat(
    agents=[code_reviewer, code_fixer],
    messages=[],
    max_round=5,
    speaker_selection_method="round_robin"
)

manager = GroupChatManager(
    groupchat=groupchat,
    llm_config={"config_list": config_list}
)

user_proxy = UserProxyAgent(name="developer",
        human_input_mode="NEVER")

user_proxy.initiate_chat(
    manager,
    message="""Review and fix this Python code:

def process_user_data(users):
    results = []
    for user in users:
        query = f"SELECT * FROM users WHERE id = {user['id']}"
        results.append(db.execute(query))
    return results
"""
)
```

9.2 Research Assistant Team

```python
import autogen
from autogen import AssistantAgent, UserProxyAgent

config_list = [{"model": "gpt-4", "api_key": "your-api-key"}]

# Create research team
researcher = AssistantAgent(
    name="researcher",
    llm_config={"config_list": config_list},
    system_message="""You research topics thoroughly.
    Search for relevant information, statistics, and sources.
    Provide well-structured findings."""
)

writer = AssistantAgent(
    name="writer",
    llm_config={"config_list": config_list},
    system_message="""You write clear, engaging content based on
        research.
    Use headings, bullet points, and keep it accessible."""
)

fact_checker = AssistantAgent(
    name="fact_checker",
    llm_config={"config_list": config_list},
    system_message="""You verify facts and claims.
    Flag any unsubstantiated claims or potential inaccuracies."""
)

# Sequential workflow
user_proxy = UserProxyAgent(name="user", human_input_mode="NEVER")

# Step 1: Research
user_proxy.initiate_chat(
    researcher,
    message="Research the impact of remote work on employee
        productivity in tech companies."
)

# Step 2: Write (with research context)
user_proxy.initiate_chat(
    writer,
    message=f"Write an article based on this research:
        {researcher.chat_messages}"
)

# Step 3: Fact-check
user_proxy.initiate_chat(
    fact_checker,
    message=f"Verify this article: {writer.chat_messages}"
)
```

9.3 Data Analysis Pipeline

```python
import autogen
from autogen import AssistantAgent, UserProxyAgent
```

```python
config_list = [{"model": "gpt-4", "api_key": "your-api-key"}]

# Analyst agent
data_analyst = AssistantAgent(
    name="data_analyst",
    llm_config={"config_list": config_list},
    system_message="""You analyze data and generate insights.
    You can execute Python code to:
    - Load and explore datasets
    - Perform statistical analysis
    - Create visualizations
    - Generate summary statistics"""
)

# Enable code execution
user_proxy = UserProxyAgent(
    name="analyst_proxy",
    human_input_mode="NEVER",
    code_execution_config={
        "work_dir": "analysis",
        "use_docker": False
    }
)

user_proxy.initiate_chat(
    data_analyst,
    message="""Analyze this sales data:

    Month,Revenue,Customers
    January,50000,120
    February,55000,135
    March,62000,150
    April,58000,145
    May,70000,180

    Calculate:
    1. Average monthly revenue
    2. Month-over-month growth rate
    3. Average customers per month
    4. Any trends you notice"""
)
```

Module 9 Summary

- **Code Review**: Multiple agents for review/fix cycle
- **Research Team**: Sequential agents for research→write→verify
- **Data Analysis**: Code execution with analysis agents

Module 10: Best Practices and Troubleshooting

10.1 Configuration Best Practices

```
# Good: Explicit configuration
config_list = [
    {
        "model": "gpt-4",
        "api_key": "your-key",
        "temperature": 0.7,
        "max_tokens": 2000
    }
]

# Better: Use environment variables
import os
config_list = [
    {
        "model": "gpt-4",
        "api_key": os.environ.get("OPENAI_API_KEY")
    }
]

# Best: Use config file
# Create O_CONFIG.json with model settings
config_list = autogen.config_list_from_json("O_CONFIG.json")
```

10.2 Common Issues and Solutions

Issue: Agents keep repeating themselves

```
# Solution: Set max_consecutive_auto_reply
user_proxy = UserProxyAgent(
    name="user",
    max_consecutive_auto_reply=3  # Limit auto-replies
)
```

Issue: Conversation too long/context issues

```
# Solution: Implement summarization
def summarize_conversation(messages, max_tokens=500):
    summary_prompt = f"Summarize this conversation in {max_tokens}
        tokens:"
    # Use LLM to summarize old messages
    return summarized_messages

# Use in long conversations
assistant.register_reply(
    trigger=lambda *args: len(args[1]) > 20,  # After 20 messages
    reply_func=lambda r, m, s, c: (True, summarize_conversation(m))
)
```

Issue: Rate limiting

```
# Solution: Add delays and retry logic
import time
```

```python
def safe_call(agent, *args, **kwargs):
    for attempt in range(3):
        try:
            return agent.generate_reply(*args, **kwargs)
        except Exception as e:
            if "rate_limit" in str(e):
                time.sleep(2 ** attempt)  # Exponential backoff
            raise
```

10.3 Security Considerations

```python
# Never expose API keys in code
# Use environment variables or config files

# Validate function inputs
def safe_calculate(expression: str) -> str:
    # Only allow safe expressions
    allowed_chars = set("0123456789+-*/.() ")
    if not all(c in allowed_chars for c in expression):
        return "Error: Invalid characters in expression"
    return eval(expression)

# Sanitize user inputs in prompts
def sanitize_input(user_input):
    # Remove potentially harmful patterns
    dangerous = ["{{", "}}", "ignore previous", "system:"]
    sanitized = user_input
    for pattern in dangerous:
        sanitized = sanitized.replace(pattern, "")
    return sanitized
```

10.4 Performance Optimization

```python
# Use caching
llm_config = {
    "config_list": config_list,
    "cache_seed": 42  # Enable caching for identical requests
}

# Parallel execution where possible
from concurrent.futures import ThreadPoolExecutor

def run_parallel_agents(agents, task):
    with ThreadPoolExecutor(max_workers=len(agents)) as executor:
        futures = [executor.submit(agent.initiate_chat, task) for
        agent in agents]
        return [f.result() for f in futures]

# Monitor token usage
def estimate_tokens(messages):
    # Rough estimate: 1 token ≈ 4 characters
    total = sum(len(m["content"]) // 4 for m in messages)
    return total
```

10.5 Testing AutoGen Applications

```python
import pytest
from autogen import AssistantAgent, UserProxyAgent

def test_agent_response():
    config_list = [{"model": "gpt-4", "api_key": "test-key"}]

    assistant = AssistantAgent(
        name="test_agent",
        llm_config={"config_list": config_list}
    )

    # Test basic response generation
    user_proxy = UserProxyAgent(name="test_user",
        human_input_mode="NEVER")

    user_proxy.initiate_chat(assistant, message="Say 'hello'")

    # Verify response exists
    messages = assistant.chat_messages[user_proxy]
    assert len(messages) > 0
    assert "hello" in messages[-1]["content"].lower()

def test_function_calling():
    def test_func(x: int) -> int:
        return x * 2

    assistant = AssistantAgent(
        name="test_agent",
        llm_config={"config_list": config_list}
    )

    assistant.register_function(function_map={"test_func": test_func})

    # Test function is registered
    assert "test_func" in assistant.function_map
```

Module 11: Production Deployment Patterns

11.1 Error Handling

```python
import logging
from autogen import AssistantAgent, UserProxyAgent

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def robust_chat(agent, recipient, message, max_retries=3):
    for attempt in range(max_retries):
        try:
            agent.initiate_chat(recipient, message)
            return True
        except Exception as e:
            logger.error(f"Attempt {attempt + 1} failed: {e}")
```

```
            if attempt == max_retries - 1:
                raise
    return False
```

11.2 State Management

```
# Persistent conversation state
import json

class StatefulAgent:
    def __init__(self, agent, state_file="agent_state.json"):
        self.agent = agent
        self.state_file = state_file
        self.state = self.load_state()

    def load_state(self):
        try:
            with open(self.state_file, "r") as f:
                return json.load(f)
        except FileNotFoundError:
            return {"messages": [], "context": {}}

    def save_state(self):
        with open(self.state_file, "w") as f:
            json.dump(self.state, f)

    def chat(self, message):
        self.agent.initiate_chat(message)
        self.state["messages"].append(message)
        self.save_state()
```

11.3 Async Support

```
import asyncio
from autogen import AssistantAgent

async def async_chat(assistant, user_proxy, message):
    # AutoGen supports async through initiate_chat
    # Wrap in asyncio for other async operations

    loop = asyncio.get_event_loop()
    await loop.run_in_executor(
        None,
        lambda: user_proxy.initiate_chat(assistant, message)
    )

# Run multiple conversations
async def run_conversations():
    tasks = [
        async_chat(assistant1, user_proxy1, "Task 1"),
        async_chat(assistant2, user_proxy2, "Task 2"),
    ]
    await asyncio.gather(*tasks)
```

Summary and Next Steps

What You've Learned

1. **Core Concepts**: Agents, conversations, messages, and LLM configuration
2. **Agent Types**: AssistantAgent, UserProxyAgent, GroupChat
3. **Tools & Functions**: Adding capabilities beyond text generation
4. **Advanced Patterns**: Nested chats, sequential workflows, speaker selection
5. **Human-in-the-Loop**: Various modes for human interaction
6. **Real-World Applications**: Code review, research teams, data analysis
7. **Best Practices**: Security, performance, testing, production patterns

AutoGen Ecosystem Resources

- **GitHub**: https://github.com/microsoft/autogen
- **Documentation**: https://microsoft.github.io/autogen/
- **Examples**: https://github.com/microsoft/autogen/tree/main/samples
- **Discord**: Join the AutoGen community for support

Next Steps for Your Learning

1. **Practice**: Build your own multi-agent application
2. **Explore**: Try Azure AI Studio for managed AutoGen
3. **Contribute**: Engage with the open-source community
4. **Deep Dive**: Study the source code for advanced customization

Recommended Projects to Build

1. **Meeting Scheduler**: Multiple agents coordinate schedules
2. **Content Pipeline**: Research → Write → Edit → Publish
3. **Customer Support**: Triage → Respond → Escalate workflow
4. **Code Assistant**: Write → Test → Review → Refactor

Quick Reference

```python
# Import
import autogen
from autogen import AssistantAgent, UserProxyAgent, GroupChat,
        GroupChatManager

# Create agents
assistant = AssistantAgent(name="assistant", llm_config=
        {"config_list": config_list})
user_proxy = UserProxyAgent(name="user", human_input_mode="NEVER")

# Start conversation
user_proxy.initiate_chat(assistant, message="Your message here")

# Add functions
assistant.register_function(function_map={"func_name": func_impl})

# Group chat
```

```
groupchat = GroupChat(agents=[agent1, agent2])
manager = GroupChatManager(groupchat=groupchat, llm_config=
        {"config_list": config_list})

# Configuration
config_list = autogen.config_list_from_json("O_CONFIG.json")
```

Course created for Alpax Prime - Build, Content, and Ops